

33회 ADP 복원

기계학습 (60점)

데이터 설명

- 데이터 출처 : <https://archive.ics.uci.edu/ml/datasets/HCV+data> (<https://archive.ics.uci.edu/ml/datasets/HCV+data>) 후처리
- 데이터 링크 : ./data/p1.csv
- 데이터 설명 : 혈액수치 데이터, 간염 정도에 따른

Category (종속변수)

- 0: 정상
- 1,2,3 (간염 정도, 심각성 : 3>2>1, 3은 간경화)

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import koreanize_matplotlib
import warnings

warnings.filterwarnings('ignore')
```

```
In [216]: import pandas as pd
d1 = pd.read_csv("https://raw.githubusercontent.com/Datamanim/datarepo/refs/heads/master/adp/33/data/p1.csv")
d1.head()
```

Out[216]:

	Category	Age	Sex	ALB	ALP	ALT	AST	BIL	CHE	CHOL	CREA	GGT	PROT
0	0=B	32	m	38.5	52.5	7.7	22.1	7.5	6.93	3.23	106.0	12.1	69.0
1	0=B	32	m	38.5	70.3	18.0	24.7	3.9	11.17	4.80	74.0	15.6	76.5
2	0=B	32	m	46.9	74.7	36.2	52.6	6.1	8.84	5.20	86.0	33.2	79.3
3	0=B	32	m	43.2	52.0	30.6	22.6	18.9	7.33	4.74	80.0	33.8	75.7
4	0=B	32	m	39.2	74.1	32.6	24.8	9.6	9.15	4.32	76.0	29.9	68.7

```
In [217]: d1.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 608 entries, 0 to 607
Data columns (total 13 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Category    608 non-null    object
1   Age         608 non-null    int64
2   Sex         608 non-null    object
3   ALB         607 non-null    float64
4   ALP         590 non-null    float64
5   ALT         607 non-null    float64
6   AST         608 non-null    float64
7   BIL         608 non-null    float64
8   CHE         608 non-null    float64
9   CHOL        598 non-null    float64
10  CREA        608 non-null    float64
11  GGT         608 non-null    float64
12  PROT        607 non-null    float64
dtypes: float64(10), int64(1), object(2)
memory usage: 61.9+ KB
```

일부 변수에서 결측치가 확인된다.

1-1 결측치 처리 방안 2개 제시하고, 처리 전후 비교 및 더 나은 방안 선택하여 전처리 완료

```
In [222]: d1.isna().mean()[d1.isna().sum(>0)*100
```

```
Out[222]: ALB      0.164474
          ALP      2.960526
          ALT      0.164474
          CHOL      1.644737
          PROT      0.164474
          dtype: float64
```

alb 등 5개 변수에서 결측치가 확인된다.

다음의 방법을 제시하고 분포를 비교하여 원본 데이터 분포와 큰 차이가 없는 방안을 선택한다.

1. 결측치 제거: 전체 데이터 대비 0.16~3% 수준의 작은 비율의 결측치이므로 삭제하는 것이 효율적일 수 있다.
원본 데이터의 왜곡을 피할 수 있으며 시간, 방법이 간편하고 빠르다.
2. 대푯값 보간: 데이터의 결측치가 5% 이상일 정도로 무시할 수 없는 수준의 결측치가 있을 때 비교적 간편하게 대푯값으로 보간할 수 있다.
여기서는 연속형 변수이므로 중앙값으로 대체한다.(중앙값 대체 시 이상치 영향 최소화 할 수 있음)

```
In [228]: cols = d1.isna().sum()[d1.isna().sum(>0)].index.values

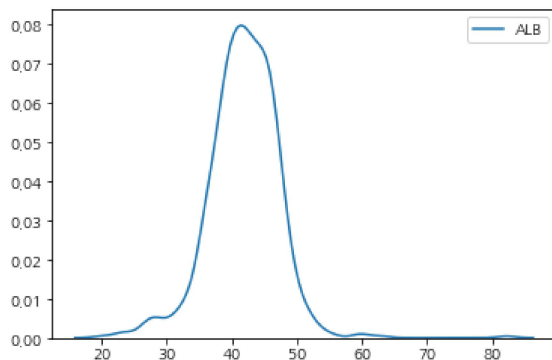
d11 = d1.dropna()

d12 = d1.copy()

for c in cols:
    d12[c] = d12[c].fillna(d12[c].median())
```

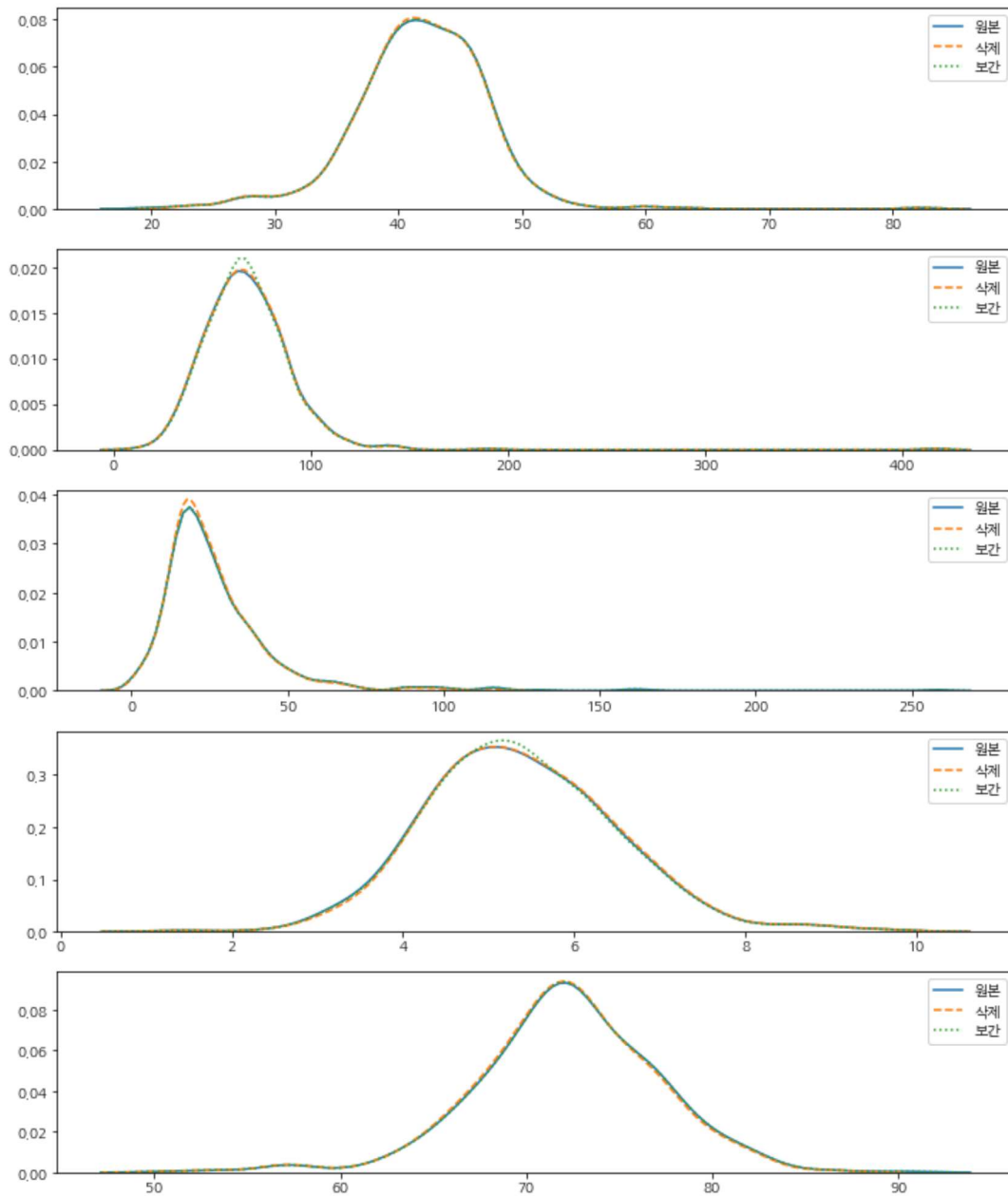
```
In [239]: sns.kdeplot(data=d1['ALB'])
```

```
Out[239]: <matplotlib.axes._subplots.AxesSubplot at 0x253c60b6a08>
```



```
In [245]: fig, ax = plt.subplots(5, 1, figsize=(12, 15))

for c, ax in zip(cols, ax.flatten()):
    sns.kdeplot(data= d1[c], ax=ax, label='원본')
    sns.kdeplot(data=d11[c], ax=ax, ls='--', label='삭제')
    sns.kdeplot(data=d12[c], ax=ax, ls='dotted', label='보간')
plt.show()
```



삭제를 했을 때 원 데이터와 분포가 매우 유사하므로(데이터 분포 왜곡 없으므로) 결측치를 삭제한다.

```
In [247]: print(f"결측치 삭제 전: {len(d1)} 개")

d1 = d1.dropna()

print(f"결측치 삭제 후: {len(d1)} 개")
```

결측치 삭제 전: 608 개
결측치 삭제 후: 582 개

1-2 이상치를 확인하고 처리 여부 판단 후 작업

```
In [250]: n_cols = d1.select_dtypes('number').columns

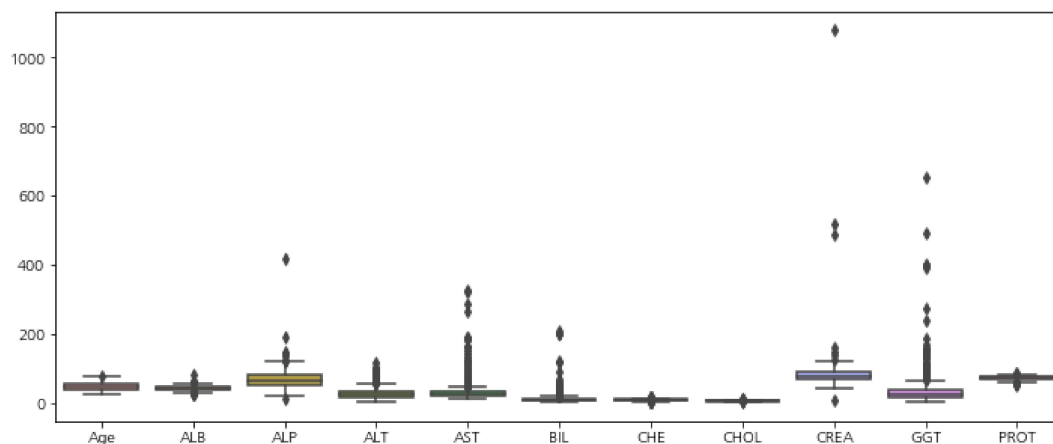
tmp = d1[n_cols].describe().T
q1 = tmp['25%']
q3 = tmp['75%']
iqr = q3 - q1
up = q3 + (1.5*iqr)
down = q1 - (1.5*iqr)

for c in up.index:
    er = d1[(d1[c] > up[c])|(d1[c] < down[c])]

    print(f"{c} 컬럼 결측치 갯수: {len(er)} ({len(er)/len(d1):.2f}%")
```

```
Age 컬럼 결측치 갯수: 1 (0.00%)
ALB 컬럼 결측치 갯수: 21 (0.04%)
ALP 컬럼 결측치 갯수: 10 (0.02%)
ALT 컬럼 결측치 갯수: 24 (0.04%)
AST 컬럼 결측치 갯수: 53 (0.09%)
BIL 컬럼 결측치 갯수: 48 (0.08%)
CHE 컬럼 결측치 갯수: 22 (0.04%)
CHOL 컬럼 결측치 갯수: 12 (0.02%)
CREA 컬럼 결측치 갯수: 8 (0.01%)
GGT 컬럼 결측치 갯수: 71 (0.12%)
PROT 컬럼 결측치 갯수: 13 (0.02%)
```

```
In [255]: plt.figure(figsize=(12, 5))
sns.boxplot(data=d1[n_cols])
plt.show()
```



iqr 방식으로 상하단 미만, 초과 임계치를 설정 후 이상치를 확인하였다.

크레아틴, GGT 등에서 상단에 높은 이상치 후보군이 보이지만 해당 데이터에 대한 사전지식이 없으므로 단순히 통계적으로 범위 바깥에 존재한다고

이상치로 확신할 수 없다.(해당 값이 유의미할 수도 있기 때문이다.)

따라서 데이터 이상치를 처리하지 않는다.(이상치 후보군 비율도 1% 미만으로 매우 낮다.)

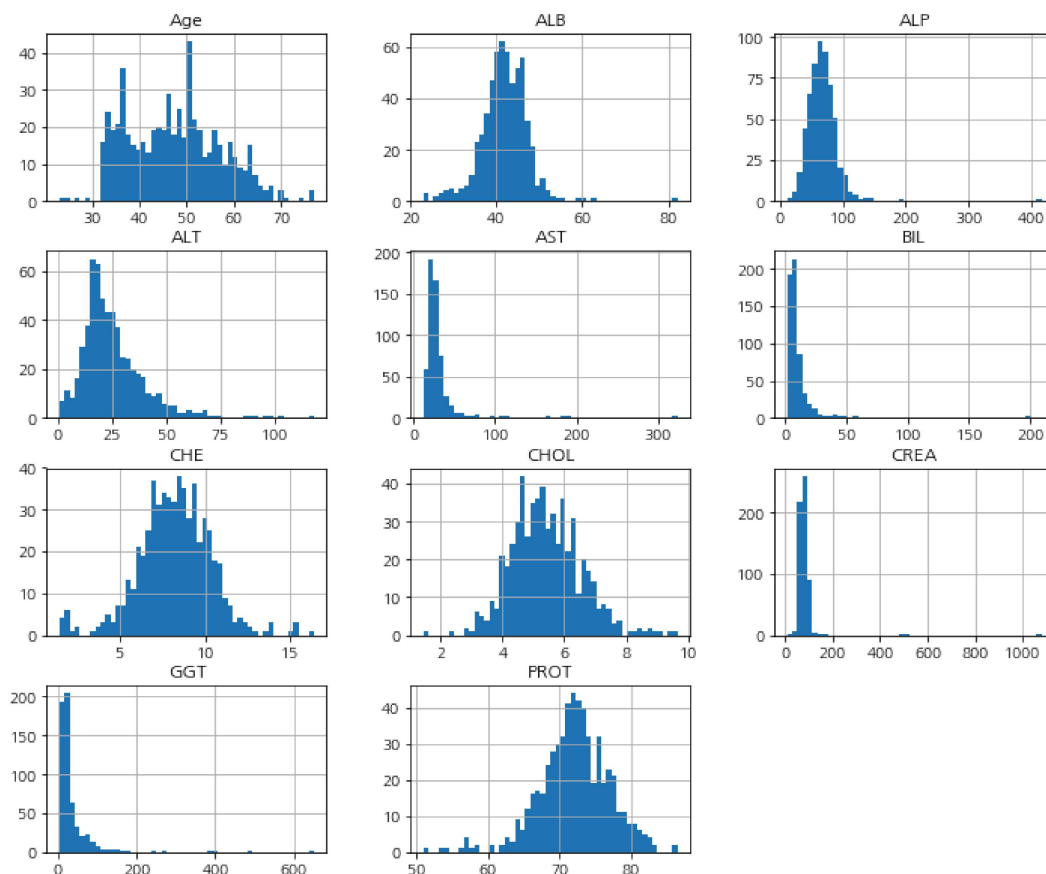
1-3 위 두 작업 적용 후 EDA 진행 및 종속변수를 기준으로 각 독립변수의 차이가 있는지 확인하라 (시각화포함)

```
In [259]: # 연속형 변수에 대한 통계량 및 시각화
n_cols = d1.select_dtypes('number').columns
c_cols = d1.select_dtypes('O').columns

display(d1[n_cols].describe().T)

d1[n_cols].hist(bins=50, layout=(4, 3), figsize=(12, 10))
plt.show()
```

	count	mean	std	min	25%	50%	75%	max
Age	582.0	47.295533	9.863890	23.00	39.000	47.000	54.0000	77.00
ALB	582.0	41.831443	5.369272	23.00	39.000	41.950	45.1750	82.20
ALP	582.0	67.651890	25.148941	11.30	52.500	66.000	79.3000	416.60
ALT	582.0	25.666838	14.920906	0.90	16.325	22.500	31.7500	118.10
AST	582.0	33.325086	32.357234	12.00	21.425	25.600	31.6000	324.00
BIL	582.0	11.094330	17.494290	1.80	5.300	7.150	11.0000	209.00
CHE	582.0	8.212337	2.151660	1.42	6.950	8.265	9.5650	16.41
CHOL	582.0	5.402646	1.114920	1.43	4.630	5.310	6.0875	9.67
CREA	582.0	81.909107	50.669976	8.00	68.000	77.000	89.0000	1079.10
GGT	582.0	36.835567	51.409352	4.50	15.450	22.650	36.2000	650.90
PROT	582.0	72.106357	4.857867	51.00	69.300	72.100	75.2000	86.50

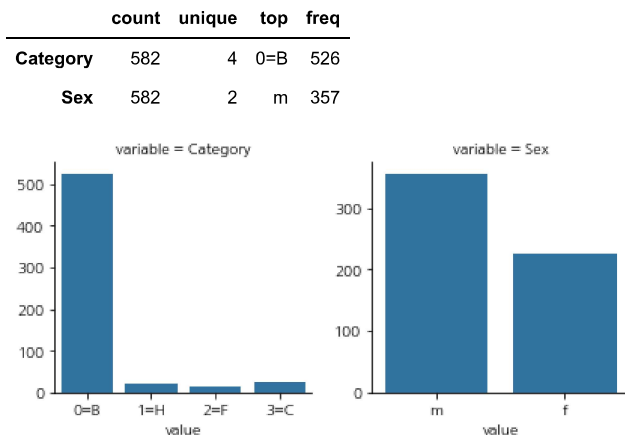


- 정규분포에 가까움: che, chol, prot
- 대부분의 변수가 양의 왜도: age, 정규분포 가까운 변수 제외

만약 분포 모양에 민감한 모델 학습 시 양의 왜도 변수는 log 화를 통한 분포모양 교정이 필요할 것이다.

```
In [260]: display(d1[c_cols].describe().T)

mt = d1[c_cols].melt()
g = sns.FacetGrid(mt, col='variable', col_wrap=2, sharex=False, sharey=False)
g.map(sns.countplot, 'value')
plt.show()
```

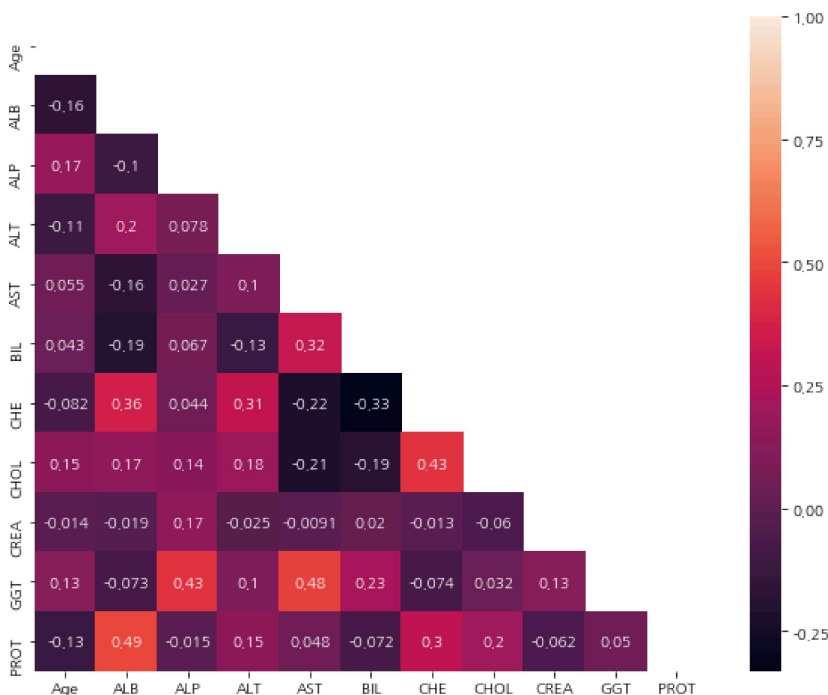


두 카테고리 변수에 대해서는 빈도 및 레벨을 확인할 수 있다.

- 카테고리는 0=B가 압도적으로 많은 비율 차이로 불균형하며,
- 성별은 남자가 다소 많으나 크게 불균형하지 않다.

```
In [261]: plt.figure(figsize=(10, 8))

corr = d1.corr()
mask = np.triu(np.ones_like(corr, dtype='bool'))
sns.heatmap(corr, mask=mask, annot=True)
plt.show()
```



피어슨 상관관계 분석결과 유의미한 1:1 상관관계는 존재하지 않는다.
보통 0.7~0.9의 절댓값을 가지는 상관계수가 있다면 유의미한 상관관계라고 한다.

더 확실한 데이터 내 상관관계 분석을 위해서는 vif를 활용할 수 있다.

1.4 주어진 데이터로부터 주성분분석을 수행해 새로운 설명변수를 도출할 수 있는지를 검토 . 판단한 뒤, 그 결과를 제시하시오

주성분분석은 svd기법을 활용하여 고유향 벡터(분산설명력)이 높은 순으로 재정렬, 일정수준의 분산설명력을 가지는 설명변수로 축소함으로써 다중공선성을 제거하고 차원이 높은 경우 저차원으로 축소함으로써 계산비용을 줄이는 효과가 있다.
다만 어느정도 데이터의 결손이 발생한다.

vif를 통해 데이터의 다중공선성 여부를 판단 후 다중공선성이 심하다면 해당 기법을 고려할 수 있다.

```
In [267]: from statsmodels.stats.outliers_influence import variance_inflation_factor

vif = pd.DataFrame()
vif['name'] = n_cols
vif['vif'] = [variance_inflation_factor(d1[n_cols].values, i) for i in range(len(n_cols))]

display(vif.sort_values(by='vif', ascending=False))
```

	name	vif
10	PROT	144.191849
1	ALB	92.804861
7	CHOL	33.561584
6	CHE	24.546356
0	Age	23.304334
2	ALP	11.071642
3	ALT	4.769372
8	CREA	3.714070
4	AST	3.363418
9	GGT	2.564956
5	BIL	1.722451

PROT 144.191849 ALB 92.804861 CHOL 33.561584 CHE 24.546356

으로 vif 10이상의 다중공선성이 확인된다. 따라서 pca를 수행하여 다중공선성을 제거한다.

```

In [281]: from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

xx = d1[n_cols]

#거리기반 방식으로 정규화 우선 수행
sc = StandardScaler()
xx = sc.fit_transform(xx)

#pca 적용
pca = PCA()
pca.fit_transform(xx)

rs = pd.DataFrame({
    "고윳값": pca.singular_values_,
    "분산설명력": pca.explained_variance_ratio_,
    "누적분산설명력": np.cumsum(pca.explained_variance_ratio_)
})

display(rs)

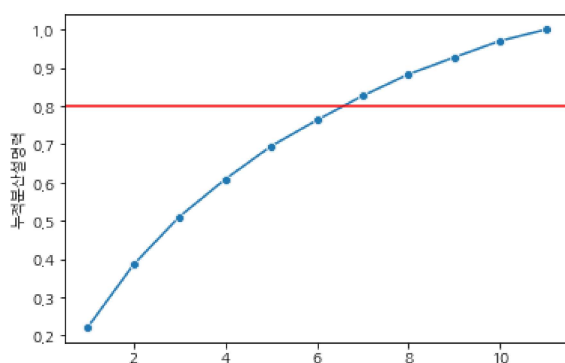
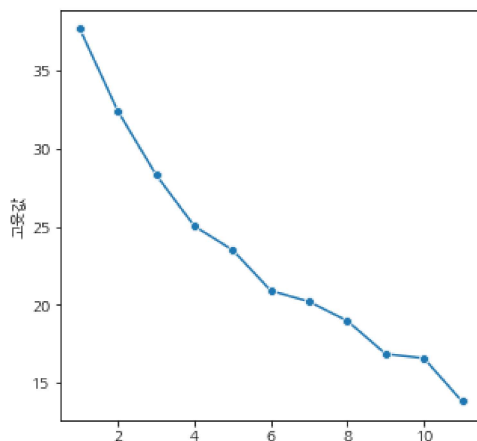
plt.figure(figsize=(5, 5))

sns.lineplot(data=rs, x=rs.index+1, y='고윳값', marker='o')
plt.show()

sns.lineplot(data=rs, x=rs.index+1, y='누적분산설명력', marker='o')
plt.axhline(y=0.8, color='r')
plt.show()

```

	고윳값	분산설명력	누적분산설명력
0	37.668811	0.221640	0.221640
1	32.385230	0.163824	0.385464
2	28.303095	0.125127	0.510592
3	25.030259	0.097862	0.608454
4	23.504640	0.086296	0.694750
5	20.896996	0.068211	0.762961
6	20.198784	0.063729	0.826689
7	18.966658	0.056191	0.882880
8	16.854372	0.044372	0.927252
9	16.579235	0.042935	0.970187
10	13.815208	0.029813	1.000000



고윳값이 모두 1 이상이며(카이저 기준), 고윳값 scree plot 상 엘보우 지점은 명확하게 관찰되지 않는다.
누적분산설명력이 0.8 이상인 7개의 주성분을 새로운 파생변수로 선택하도록 한다.

2-1 데이터 불균형이 왜 문제인지 설명하고 불균형 해결방법에 대해 두가지를 설명하라

앞에서 종속변수인 카테고리리는 불균형 정도가 심했다.

데이터가 불균형이면 majority class에 집중되어 학습이 되므로 minority class를 오분류하는 문제가 발생한다.

실제 minor를 majority class로 오분류하는 문제(recall 점수 낮음-2중 오류와 연관)

만약 암을 분류하는 분류모델에서 암 환자 비율이 낮아 실제 암인데도 정상으로 판별한다면 해당 환자는 치료시기를 놓쳐서 사망할 수도 있기 때문이다.

해결방법은 다음과 같다.

- 오버샘플링: 마이너 클래스를 메이저 클래스에 맞춰 증강시키는 방식이다. 여기서는 smote를 사용하여 과적합을 어느정도 예방할 수 있다.
- 앙상블 기법 사용: 배깅, 부스팅 기반 특히 트리모델은 불균형 데이터에 비교적 강건한 특성을 가지고 있다.(불순도 기반으로 트리분할)

2-2 불균형 데이터 분류모델에 적합한 평가지표 3개 제시하고 그 이유 설명하라

- f1 score: 정밀도와 재현율이 상충 관계일 때 두 지표의 균형점을 한 값으로 평가할 수 있다.
특히 소수클래스 예측을 놓치는 경우를 모두 반영하므로 비교적 공정한 단일지표로 유용함
- pr_auc: 정밀도-재현율 곡선 아래면적을 의미하며, 정밀도와 재현율이 상충 관계에서 불균형 데이터에서 모델 성능평가 시 유용
- roc_curve의 auc: tpr, fpr 곡선의 면적으로 실제 데이터 레이블 비정상 기준 recall과 레이블1을 오분류할 경우를 고려가능

모두 1에 가까울 수록 높은 성능을 가짐(macro로 각 레이블 평균치 비교)

2-3 종속 변수를 정상(Category 0)과 비정상(Category 1,2,3)로 변경한 후 위의 2가지 불균형 처리 방법을 사용하고, 3가지 평가지표로 결과를 각각 평가하라.

(학습 규칙)

- 종속 변수를 이진화 후 불균형 처리 방법 진행
- 전처리 후 데이터 행의 index number를 초기화 하라(0부터 시작)
- index number 값이 5로 나눈 나머지가 0이라면 test, 그 외의 경우 train set으로 하여 진행
- 3가지 분류 모델을 사용하라

```
In [283]: d1['Category'].value_counts()
```

```
Out[283]: 0=B      526
          3=C       24
          1=H       20
          2=F       12
          Name: Category, dtype: int64
```

```
In [316]: x.dtypes
```

```
Out[316]: Age      int64
          Sex      object
          ALB      float64
          ALP      float64
          ALT      float64
          AST      float64
          BIL      float64
          CHE      float64
          CHOL      float64
          CREA      float64
          GGT      float64
          PROT      float64
          dtype: object
```

```

In [321]: d2 = d1.copy()

d2['Category'] = np.where(d2['Category'] == 'O=B', 0, 1)

x = d2.drop(columns=['Category'])
y = d2['Category']

from imblearn.over_sampling import SMOTENC

sm = SMOTENC(random_state=42, categorical_features=[1, 2])

x1, y1 = sm.fit_resample(x, y)

tmp1 = pd.DataFrame(x1, columns=x.columns).astype(x.dtypes)
tmp1['Category'] = y1.astype('int64')

tmp1.head()

```

Out[321]:

	Age	Sex	ALB	ALP	ALT	AST	BIL	CHE	CHOL	CREA	GGT	PROT	Category
0	32	m	38.5	52.5	7.7	22.1	7.5	6.93	3.23	106.0	12.1	69.0	0
1	32	m	38.5	70.3	18.0	24.7	3.9	11.17	4.80	74.0	15.6	76.5	0
2	32	m	46.9	74.7	36.2	52.6	6.1	8.84	5.20	86.0	33.2	79.3	0
3	32	m	43.2	52.0	30.6	22.6	18.9	7.33	4.74	80.0	33.8	75.7	0
4	32	m	39.2	74.1	32.6	24.8	9.6	9.15	4.32	76.0	29.9	68.7	0

```

In [361]: from collections import Counter

print(Counter(tmp1['Category']))

```

Counter({0: 526, 1: 526})

```
In [333]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.decomposition import PCA
from sklearn.compose import make_column_transformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import f1_score, roc_auc_score, average_precision_score

train = tmp1[tmp1.index % 5 == 0]
test = tmp1[tmp1.index % 5 != 0]

train_x, train_y = train.drop(columns=['Category']), train['Category']
test_x, test_y = test.drop(columns=['Category']), test['Category']

n_cols = train_x.select_dtypes('number').columns
c_cols = train_x.select_dtypes('O').columns

pre = make_column_transformer(
    (Pipeline([('scr', StandardScaler()), ('pca', PCA(n_components=0.8))]), n_cols),
    (Pipeline([('one', OneHotEncoder(handle_unknown='ignore'))]), c_cols),
    remainder='passthrough'
)

lg = Pipeline([
    ('pre', pre),
    ('cl', LogisticRegression(random_state=42))
])

svm = Pipeline([
    ('pre', pre),
    ('cl', SVC(random_state=42, probability=True))
])

rf = Pipeline([
    ('pre', pre),
    ('cl', RandomForestClassifier(random_state=42))
])

m_list = [('로지스틱', lg), ('svm', svm), ('랜덤포레스트', rf)]
d_list = [('train', train_x, train_y), ('test', test_x, test_y)]

rs = pd.DataFrame(columns = ['model', 'data', 'f1', 'auc', 'pr_auc'])

for n1, model in m_list:
    model.fit(train_x, train_y)
    for n2, x, y in d_list:
        pred = model.predict(x)
        f1 = f1_score(y, pred, average='macro')
        auc = roc_auc_score(y, pred, average='macro')
        pr = average_precision_score(y, pred, average='macro')
        rs.loc[len(rs)] = [n1, n2, f1, auc, pr]

display(rs)
```

	model	data	f1	auc	pr_auc
0	로지스틱	train	0.933636	0.933603	0.907910
1	로지스틱	test	0.906062	0.906071	0.870109
2	svm	train	0.976303	0.976325	0.962668
3	svm	test	0.958364	0.958359	0.931232
4	랜덤포레스트	train	1.000000	1.000000	1.000000
5	랜덤포레스트	test	0.946462	0.946465	0.914097

각 모델 학습 결과이다. 3개의 모델을 가지고 학습하였다.

모든 모델에는 smote를 적용하여 데이터를 증강하였다.

세 모델 모두에서 과적합은 일어나지 않았다. 또한 성능도 준수한 편이다. smote를 통한 증강이 모델 성능 개선에 영향을 주었음을 알 수 있다.

테스트 셋 기준으로 랜덤포레스트의 결과가 가장 좋았는데, 배경기반 앙상블 모델이며 트리에서 파생된 모델이므로 불균형 데이터에 강건한 특성을 보여주고 있다.

3-1 1에서 전처리 완료한 데이터를 바탕으로 종속변수에서 0을 제외한 1,2,3의 데이터만 필터하여 분류모델 학습 및 평가를 진행하라.

-> 셋 이상의 범주(class)**를 가지는 문제는 다중 분류(Multi-class Classification) 문제 (학습 규칙)

- 전처리 후 데이터 행의 index number를 초기화 하라(0부터 시작)
- index number 값이 5로 나눈 나머지가 0이라면 test, 그 외의 경우 train set으로 하여 진행
- 3개의 분류 모델을 사용하라

```
In [334]: d1['Category'].value_counts()
```

```
Out[334]: 0=B    526
          3=C     24
          1=H     20
          2=F     12
          Name: Category, dtype: int64
```

```

In [346]: from sklearn.metrics import accuracy_score, classification_report

tmp2 = d1[d1['Category'] != '0=B']
tmp2['Category'] = tmp2['Category'].str[:1].astype('int64')

tmp2 = tmp2.reset_index(drop=True)

train = tmp2[tmp2.index % 5 == 0]
test = tmp2[tmp2.index % 5 != 0]

train_x, train_y = train.drop(columns=['Category']), train['Category']
test_x, test_y = test.drop(columns=['Category']), test['Category']

n_cols = train_x.select_dtypes('number').columns
c_cols = train_x.select_dtypes('O').columns

pre = make_column_transformer(
    (Pipeline([['scr', StandardScaler()], ('pca', PCA(n_components=0.8))]), n_cols),
    (Pipeline([['one', OneHotEncoder(handle_unknown='ignore')]), c_cols),
    remainder='passthrough'
)

lg = Pipeline([
    ('pre', pre),
    ('cl', LogisticRegression(random_state=42))
])

svm = Pipeline([
    ('pre', pre),
    ('cl', SVC(random_state=42, probability=True))
])

rf = Pipeline([
    ('pre', pre),
    ('cl', RandomForestClassifier(random_state=42))
])

m_list = [('로지스틱', lg), ('svm', svm), ('랜덤포레스트', rf)]

d_list = [('train', train_x, train_y), ('test', test_x, test_y)]

rs = pd.DataFrame(columns = ['model', 'data', 'f1', 'auc', 'acc'])

for n1, model in m_list:
    model.fit(train_x, train_y)
    for n2, x, y in d_list:
        pred = model.predict(x)
        proba = model.predict_proba(x)
        f1 = f1_score(y, pred, average='macro')
        auc = roc_auc_score(y, proba, average='macro', multi_class='ovo')
        acc = accuracy_score(y, pred)
        rs.loc[len(rs)] = [n1, n2, f1, auc, acc]
        print(classification_report(y, pred))

display(rs)

```

	precision	recall	f1-score	support
1	1.00	0.75	0.86	4
2	0.75	1.00	0.86	3
3	1.00	1.00	1.00	5
accuracy			0.92	12
macro avg	0.92	0.92	0.90	12
weighted avg	0.94	0.92	0.92	12

	precision	recall	f1-score	support
1	0.50	0.44	0.47	16
2	0.33	0.44	0.38	9
3	0.94	0.89	0.92	19
accuracy			0.64	44
macro avg	0.59	0.59	0.59	44
weighted avg	0.66	0.64	0.64	44

	precision	recall	f1-score	support
1	0.80	1.00	0.89	4
2	1.00	0.67	0.80	3
3	1.00	1.00	1.00	5
accuracy			0.92	12
macro avg	0.93	0.89	0.90	12
weighted avg	0.93	0.92	0.91	12

	precision	recall	f1-score	support
1	0.50	0.69	0.58	16
2	0.43	0.33	0.38	9
3	0.93	0.74	0.82	19
accuracy			0.64	44
macro avg	0.62	0.59	0.59	44
weighted avg	0.67	0.64	0.64	44

	precision	recall	f1-score	support
1	1.00	1.00	1.00	4
2	1.00	1.00	1.00	3
3	1.00	1.00	1.00	5
accuracy			1.00	12
macro avg	1.00	1.00	1.00	12
weighted avg	1.00	1.00	1.00	12

	precision	recall	f1-score	support
1	0.57	0.50	0.53	16
2	0.25	0.22	0.24	9
3	0.77	0.89	0.83	19
accuracy			0.61	44
macro avg	0.53	0.54	0.53	44
weighted avg	0.59	0.61	0.60	44

	model	data	f1	auc	acc
0	로지스틱	train	0.904762	0.986111	0.916667
1	로지스틱	test	0.588846	0.720517	0.636364
2	svm	train	0.896296	0.680556	0.916667
3	svm	test	0.592492	0.830897	0.636364
4	랜덤포레스트	train	1.000000	1.000000	1.000000
5	랜덤포레스트	test	0.532632	0.741106	0.613636

불균형을 유발했던 0레이블을 제거하여 클래스 분포를 확인한 결과 균형잡힌 데이터로 변환되어 정확도를 주요 지표로 채택하되, F1-score 등 보조 지표도 함께 고려한다. (pr_auc는 미사용)

세 모델 모두에서 과적합이 발생했다. train/test 셋의 분포가 다를 수 있음이 예상된다.

또한 데이터 수가 충분하지 않았던 것도 원인이 될 수 있다.

세 모델에서 테스트 셋 기준으로 svm 모델의 성능이 가장 높았으므로 최적의 모델로 선정하나 과적합 해결을 위한 증화 추출, 데이터 증강 후 가중치 학습, 교차검증, 정규화 등의 방법이 고려된다.

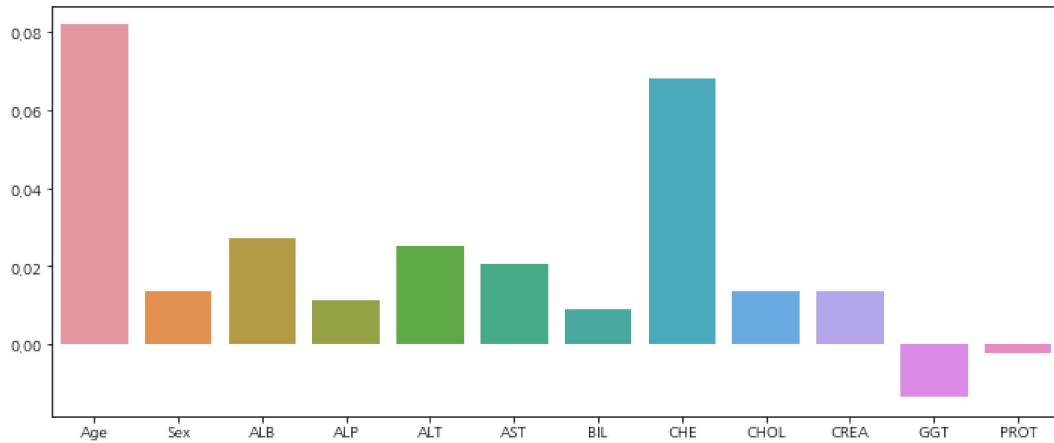
3-2 학습한 모델 결과와 평가지표를 바탕으로 간염여부에 영향을 미치는 요소들에 대해 논의하라

```
In [347]: from sklearn.inspection import permutation_importance

imp = permutation_importance(svm, test_x, test_y, n_repeats=10, random_state=42)
feat = train_x.columns

f_mean = imp.importances_mean

plt.figure(figsize=((12, 5)))
sns.barplot(y=f_mean, x=feat)
plt.show()
```



퍼뮤테이션 임포턴스로 test set 기준 변수별 모델학습 기여도를 측정하였다.

나이, che가 모델 학습에 큰 영향을 주며, ggt, prot는 모델 학습에 악영향을 끼치는 것을 알 수 있다.

```

In [359]: from sklearn.inspection import plot_partial_dependence

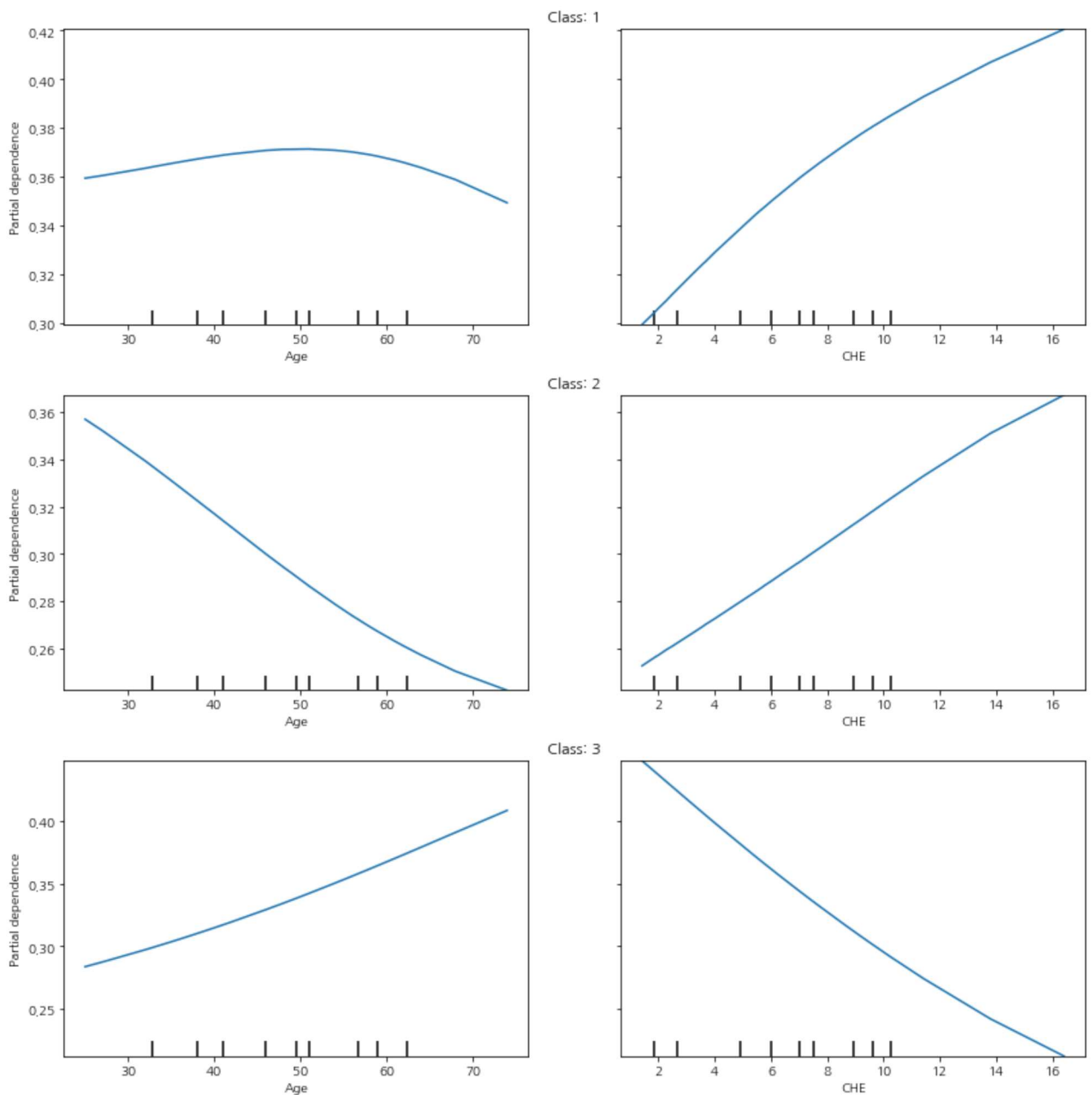
s_cols = ['Age', 'CHE']

# 클래스 수에 맞게 서브플롯 생성
n_classes = len(svm.classes_)
fig, axes = plt.subplots(n_classes, 1, figsize=(12, 4*n_classes))

for i, class_name in enumerate(svm.classes_):
    plot_partial_dependence(estimator=svm,
                           X=test_x,
                           features=s_cols,
                           percentiles=(0, 1),
                           target=class_name, # 클래스 인덱스
                           ax=axes[i])
    axes[i].set_title(f'Class: {class_name}')

plt.tight_layout()
plt.show()

```



각 레이블 별 해당 변수의 영향을 확인한다.

- 레이블1: 나이는 전 범위에 걸쳐 있음, che가 높을수록 확률이 높아짐
- 레이블2: 나이가 어릴수록 확률이 높으며, che가 높을수록 확률이 높아짐
- 레이블3: 나이가 많을수록 확률이 높으며, che가 낮을수록 확률이 높아짐

따라서 고령자, che가 낮은 인원, 청-장년 층 혈액검사 결과 che가 높은 인원을 대상으로 간경화 검사 및 예방 프로그램을 기획한다.

4. 데이터 설명

- 데이터 출처 : <https://www.data.go.kr/data/15048032/fileData.do> (<https://www.data.go.kr/data/15048032/fileData.do>) / 후처리, 통계청 날씨정보
- 데이터 링크 :
 - ./data/p2_sb_2021.csv
 - ./data/p2_sb_2022.csv
 - ./data/p2_sb_2023.csv
 - ./data/p2_w_2021.csv
 - ./data/p2_w_2022.csv
 - ./data/p2_w_2023.csv
- 데이터 설명 :
 - 21년~23년의 J시의 2개 지하철호선 지하철의 시간대별 승하차 인원 정보 데이터 (p2_sb_년도.csv)
 - X5~X22 : 5시부터 22시 승차 또는 하차 인원, (X5 : 05시의 계절 날짜와 대응)
 - 역번호 : 각 정류장의 고유 번호
 - 승하차구분 : 승차 또는 하차
 - 21년~23년의 J시의 특정지역구의 시간대별 계절성 정보 데이터 (p2_w_년도.csv)
 - 시간별 J시의 날씨정보

```
In [156]: pd.set_option('display.max_columns', None)
```

```
In [170]: import pandas as pd
p1 = pd.read_csv('https://raw.githubusercontent.com/Datamanim/datarepo/refs/heads/master/adp/33/data/p2_sb_2021.csv')
p2 = pd.read_csv('https://raw.githubusercontent.com/Datamanim/datarepo/refs/heads/master/adp/33/data/p2_sb_2022.csv')
p3 = pd.read_csv('https://raw.githubusercontent.com/Datamanim/datarepo/refs/heads/master/adp/33/data/p2_sb_2023.csv')
```

```
In [171]: w1 = pd.read_csv('https://raw.githubusercontent.com/Datamanim/datarepo/refs/heads/master/adp/33/data/p2_w_2021.csv')
w2 = pd.read_csv('https://raw.githubusercontent.com/Datamanim/datarepo/refs/heads/master/adp/33/data/p2_w_2022.csv')
w3 = pd.read_csv('https://raw.githubusercontent.com/Datamanim/datarepo/refs/heads/master/adp/33/data/p2_w_2023.csv')
```

```
In [172]: display(p1.head())
display(p2.head())
display(p3.head())
display(w1.head())
display(w2.head())
display(w3.head())
```

	날짜	호선	역번호	승하차구분	X5	X6	X7	X8	X9	X10	X11	X12	X13	X14	X15	X16	X17	X18	X19	X20	X21	X22
0	2021-01-01	A	201	승차	22.0	25	61	64	58	53	94	107	109	149	196	168	178	177	117	138	90	58
1	2021-01-01	A	201	하차	12.0	49	87	142	101	110	84	110	105	125	114	86	119	63	26	25	62	18
2	2021-01-01	A	202	승차	43.0	53	70	119	94	103	176	295	331	323	437	447	881	531	335	441	403	137
3	2021-01-01	A	202	하차	21.0	154	238	727	425	356	377	351	330	342	321	333	331	163	119	110	137	47
4	2021-01-01	A	203	승차	21.0	27	48	81	62	60	116	204	156	197	197	203	213	190	145	208	135	76

	날짜	호선	역번호	승하차구분	X5	X6	X7	X8	X9	X10	X11	X12	X13	X14	X15	X16	X17	X18	X19	X20	X21	X22
0	2022-01-01	A	201	승차	16.0	36	55	100	92	96	209	297	268	287	389	443	424	385	240	358	359	115
1	2022-01-01	A	201	하차	9.0	88	196	209	150	136	154	278	253	311	249	223	196	161	84	103	130	65
2	2022-01-01	A	202	승차	41.0	39	79	124	168	200	415	660	607	752	796	977	1377	1200	922	985	1094	417
3	2022-01-01	A	202	하차	38.0	156	220	707	564	529	630	762	776	801	828	837	861	649	328	245	403	176
4	2022-01-01	A	203	승차	24.0	36	72	70	105	263	280	554	439	413	409	395	497	475	377	581	689	212

	날짜	호선	역번호	승하차구분	X5	X6	X7	X8	X9	X10	X11	X12	X13	X14	X15	X16	X17	X18	X19	X20	X21	X22
0	2023-01-01	A	201	승차	43.0	43	43	138	140	287	392	602	482	527	601	627	761	801	581	507	503	304
1	2023-01-01	A	201	하차	36.0	126	144	245	283	348	332	418	403	432	450	371	354	308	227	232	237	208
2	2023-01-01	A	202	승차	108.0	111	120	175	278	556	881	1097	1154	1258	1372	1556	1687	1901	1347	1333	1162	1029
3	2023-01-01	A	202	하차	114.0	312	387	869	899	773	1121	1106	1252	1236	1145	1319	1289	1082	874	732	612	414
4	2023-01-01	A	203	승차	52.0	69	107	238	244	553	673	801	880	806	774	847	748	714	554	508	423	366

	일시	기온	강수량	풍속	습도	증기압	현지기압	전운량	중하층운량	지면온도
0	2021-01-01 00:00	-8.3	NaN	1.8	66	2.2	1016.4	0.0	0	-6.8
1	2021-01-01 01:00	-8.7	NaN	2.4	68	2.2	1016.4	0.0	0	-6.9
2	2021-01-01 02:00	-9.1	NaN	1.6	69	2.1	1016.2	0.0	0	-7.1
3	2021-01-01 03:00	-9.3	NaN	1.1	70	2.1	1016.8	0.0	0	-7.3
4	2021-01-01 04:00	-9.3	NaN	0.3	71	2.2	1016.2	0.0	0	-7.5

	일시	기온	강수량	풍속	습도	증기압	현지기압	전운량	중하층운량	지면온도
0	2022-01-01 00:00	-8.5	NaN	1.9	41	1.3	1021.9	0	0	-7.0
1	2022-01-01 01:00	-9.2	NaN	1.8	42	1.3	1022.2	0	0	-7.2
2	2022-01-01 02:00	-9.5	NaN	1.2	43	1.3	1022.3	0	0	-7.5
3	2022-01-01 03:00	-9.3	NaN	1.4	46	1.4	1022.6	0	0	-7.6
4	2022-01-01 04:00	-9.6	NaN	1.7	48	1.4	1021.9	0	0	-7.6

	일시	기온	강수량	풍속	습도	증기압	현지기압	전운량	중하층운량	지면온도
0	2023-01-01 00:00	0.9	NaN	1.4	72	4.7	1018.5	9	9	-1.6
1	2023-01-01 01:00	1.5	NaN	1.9	71	4.9	1018.0	6	6	-1.4
2	2023-01-01 02:00	1.5	NaN	1.9	72	4.9	1017.8	9	8	-1.6
3	2023-01-01 03:00	1.6	NaN	1.6	74	5.1	1018.1	8	8	-1.8
4	2023-01-01 04:00	1.5	NaN	1.4	74	5.0	1018.1	9	9	-1.3

```
In [183]: pv = pd.concat([p1, p2, p3], axis=0)
w = pd.concat([w1, w2, w3], axis=0)
```

```
In [184]: pv.info()

<class 'pandas.core.frame.DataFrame'>
Int64Index: 182790 entries, 0 to 61019
Data columns (total 22 columns):
#   Column      Non-Null Count  Dtype
---  -
0   날짜        182790 non-null  object
1   호선        182790 non-null  object
2   역번호      182790 non-null  int64
3   승하차구분  182790 non-null  object
4   X5          182766 non-null  float64
5   X6          182790 non-null  int64
6   X7          182790 non-null  int64
7   X8          182790 non-null  int64
8   X9          182790 non-null  int64
9   X10         182790 non-null  int64
10  X11         182790 non-null  int64
11  X12         182790 non-null  int64
12  X13         182790 non-null  int64
13  X14         182790 non-null  int64
14  X15         182790 non-null  int64
15  X16         182790 non-null  int64
16  X17         182790 non-null  int64
17  X18         182790 non-null  int64
18  X19         182790 non-null  int64
19  X20         182790 non-null  int64
20  X21         182790 non-null  int64
21  X22         182790 non-null  int64
dtypes: float64(1), int64(18), object(3)
memory usage: 32.1+ MB
```

```
In [185]: w.info()

<class 'pandas.core.frame.DataFrame'>
Int64Index: 26280 entries, 0 to 8759
Data columns (total 10 columns):
#   Column      Non-Null Count  Dtype
---  -
0   일시        26280 non-null  object
1   기온        26280 non-null  float64
2   강수량      2975 non-null   float64
3   풍속        26241 non-null  float64
4   습도        26280 non-null  int64
5   증기압      26280 non-null  float64
6   현지기압    26280 non-null  float64
7   전운량      26278 non-null  float64
8   중하층운량  26280 non-null  int64
9   지면온도    26273 non-null  float64
dtypes: float64(7), int64(2), object(1)
memory usage: 2.2+ MB
```

각 분류별 데이터를 병합한다.

4-1 아래 조건들을 참고하여 전처리를 시행하고 훈련데이터에 대한 기초통계량을 구하시오

- 승하차 데이터에서 X5~X22 모든컬럼이 0의 값을 가지는 경우 해당 날짜,호선,역번호 승하차 행은 제외한다.
- 승하차 데이터를 날짜,호선,역번호,시간대별(05~22시) 승객수의 총합을 구하여라(승차 + 하차), 결측치는 0명으로 간주한다. 컬럼명은 users로 변경한다.
- 날짜 데이터의 강수 데이터는 결측치를 0, 그 외 컬럼의 결측치는 직전값으로 대체한다.
- 날짜데이터와 일자,시간별 승하차인원 데이터를 동일날짜,시간에 대해 합친다.
- 일자정보를 바탕으로 주말 여부를 나타내는 weekday컬럼을 추가한다. (주말=1,평일=0)
- 21년,22년 데이터는 훈련데이터, 23년 데이터는 테스트 데이터로 사용한다.

```
In [186]: # 승하차 데이터에서 X5~X22 모든컬럼이 0의 값을 가지는 경우 해당 날짜,호선,역번호 승하차 행은 제외한다.

cols = pv.columns[4:]
pv = pv[~(pv[cols] == 0).all(axis=1)]
```

```
In [187]: #승하차 데이터를 날짜,호선,역번호,시간대별(05~22시) 승객수의 총합을 구하여라(승차 + 하차), 결측치는 0명으로 간주한다.
#컬럼명은 users로 변경한다.

pv[cols] = pv[cols].fillna(0)
pv = pv.groupby(['날짜', '호선', '역번호'])[cols].sum().reset_index()

pv = pv.melt(id_vars = pv.columns[:2], value_vars=cols, var_name='시간', value_name = 'users')
pv['시간'] = pv['시간'].str.replace('X', '').astype('int64')

pv.head()
```

```
Out[187]:
```

	날짜	호선	시간	users
0	2021-01-01	A	5	34.0
1	2021-01-01	A	5	64.0
2	2021-01-01	A	5	39.0
3	2021-01-01	A	5	23.0
4	2021-01-01	A	5	67.0

```
In [188]: # 날짜 데이터의 강수 데이터는 결측치를 0, 그 외 컬럼의 결측치는 직전값으로 대체한다.
col2 = w.columns[3:]

w['강수량'] = w['강수량'].fillna(0)

w[col2] = w[col2].fillna(method='ffill')

w.isna().sum()
```

```
Out[188]:
```

일시	0
기온	0
강수량	0
풍속	0
습도	0
증기압	0
현지기압	0
전운량	0
중하층운량	0
지면온도	0
dtype:	int64

```
In [189]: pv.isna().sum()
```

```
Out[189]:
```

날짜	0
호선	0
시간	0
users	0
dtype:	int64

```
In [190]: w.head()
```

```
Out[190]:
```

	일시	기온	강수량	풍속	습도	증기압	현지기압	전운량	중하층운량	지면온도
0	2021-01-01 00:00	-8.3	0.0	1.8	66	2.2	1016.4	0.0	0	-6.8
1	2021-01-01 01:00	-8.7	0.0	2.4	68	2.2	1016.4	0.0	0	-6.9
2	2021-01-01 02:00	-9.1	0.0	1.6	69	2.1	1016.2	0.0	0	-7.1
3	2021-01-01 03:00	-9.3	0.0	1.1	70	2.1	1016.8	0.0	0	-7.3
4	2021-01-01 04:00	-9.3	0.0	0.3	71	2.2	1016.2	0.0	0	-7.5

```
In [191]: w['일시'] = pd.to_datetime(w['일시'], format='%Y-%m-%d %H')

w['날짜'] = w['일시'].dt.strftime('%Y-%m-%d')
w['시간'] = w['일시'].dt.hour.astype('int64')

w.drop(columns='일시', inplace=True)
w.head()
```

```
Out[191]:
```

	기온	강수량	풍속	습도	증기압	현지기압	전운량	중하층운량	지면온도	날짜	시간
0	-8.3	0.0	1.8	66	2.2	1016.4	0.0	0	-6.8	2021-01-01	0
1	-8.7	0.0	2.4	68	2.2	1016.4	0.0	0	-6.9	2021-01-01	1
2	-9.1	0.0	1.6	69	2.1	1016.2	0.0	0	-7.1	2021-01-01	2
3	-9.3	0.0	1.1	70	2.1	1016.8	0.0	0	-7.3	2021-01-01	3
4	-9.3	0.0	0.3	71	2.2	1016.2	0.0	0	-7.5	2021-01-01	4

```
In [192]: print(len(w), len(pv))
```

```
26280 1636236
```

```
In [193]: #날짜데이터와 일자,시간별 승하차인원 데이터를 동일날짜,시간에 대해 합친다.
mg = pd.merge(pv, w, on=['날짜', '시간'], how='left', sort=['날짜', '시간'])

#일자정보를 바탕으로 주말 여부를 나타내는 weekday컬럼을 추가한다. (주말=1,평일=0)
mg['날짜'] = pd.to_datetime(mg['날짜'], format='%Y-%m-%d')
mg['weekday'] = np.where(mg['날짜'].dt.weekday.isin([5, 6]), 1, 0)

print(mg['weekday'].value_counts())
```

```
0    1167120
1     469116
Name: weekday, dtype: int64
```

```
In [194]: print(len(mg))
```

```
1636236
```

```
In [195]: #21년,22년 데이터는 훈련데이터, 23년 데이터는 테스트 데이터로 사용한다.
train, test = mg.loc[mg['날짜'].dt.year.isin(['2021', '2022'])], mg.loc[mg['날짜'].dt.year.isin(['2023'])]

print(train.shape, test.shape)
```

```
(1090620, 14) (545616, 14)
```

```
In [196]: train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 1090620 entries, 0 to 1090619
Data columns (total 14 columns):
 #   Column      Non-Null Count  Dtype
---  -
0   날짜        1090620 non-null  datetime64[ns]
1   호선        1090620 non-null  object
2   시간        1090620 non-null  int64
3   users       1090620 non-null  float64
4   기온        1090620 non-null  float64
5   강수량      1090620 non-null  float64
6   풍속        1090620 non-null  float64
7   습도        1090620 non-null  int64
8   증기압      1090620 non-null  float64
9   현지기압    1090620 non-null  float64
10  전운량      1090620 non-null  float64
11  중하층운량  1090620 non-null  int64
12  지면온도    1090620 non-null  float64
13  weekday     1090620 non-null  int32
dtypes: datetime64[ns](1), float64(8), int32(1), int64(3), object(1)
memory usage: 120.7+ MB
```

```
In [197]: #기초통계량

c_cols = '호선'
n_cols = train.select_dtypes('number').columns

display(train[n_cols].describe().T)
display(train[c_cols].describe().T)
```

	count	mean	std	min	25%	50%	75%	max
시간	1090620.0	13.500000	5.188130	5.0	9.000	13.5	18.0	22.0
users	1090620.0	2093.877278	2346.067281	0.0	597.000	1347.0	2732.0	26183.0
기온	1090620.0	14.176355	11.175155	-18.5	6.000	15.4	23.6	36.3
강수량	1090620.0	0.171750	1.440923	0.0	0.000	0.0	0.0	64.7
풍속	1090620.0	2.481560	1.134005	0.0	1.700	2.3	3.1	8.6
습도	1090620.0	62.749391	18.919553	17.0	48.000	62.0	78.0	100.0
증기압	1090620.0	12.654094	8.796690	0.7	5.375	9.9	19.7	35.2
현기압	1090620.0	1005.940944	8.094596	984.1	999.400	1006.3	1012.3	1025.8
전운량	1090620.0	4.974505	3.956197	0.0	0.000	6.0	9.0	10.0
중하층운량	1090620.0	3.064764	3.459133	0.0	0.000	1.0	7.0	10.0
지면온도	1090620.0	16.311941	13.159712	-12.7	5.100	17.1	25.7	59.8
weekday	1090620.0	0.286301	0.452032	0.0	0.000	0.0	1.0	1.0

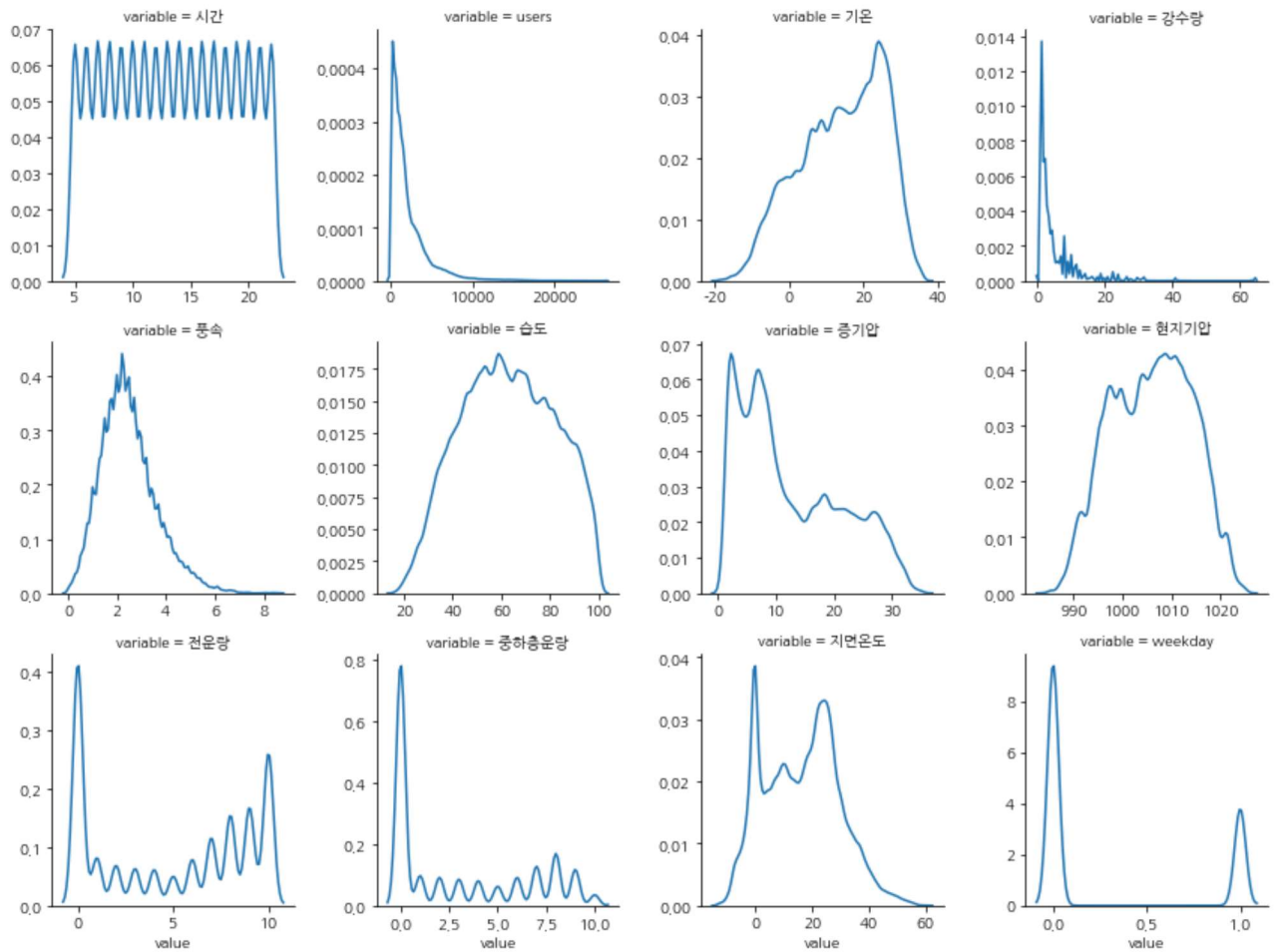
```
count    1090620
unique         2
top          A
freq        657000
Name: 호선, dtype: object
```

다음과 같이 기초통계량을 구하였다.

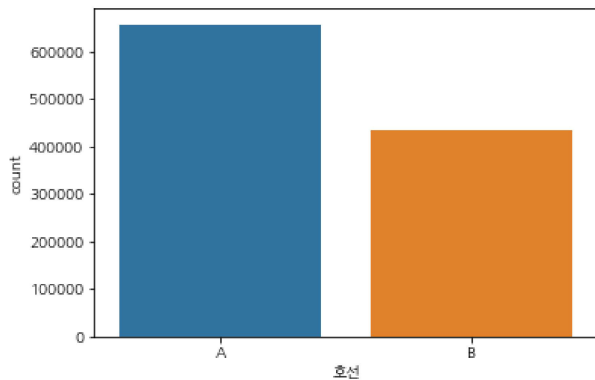
명목형 변수의 레벨은 2이며, A의 비율이 가장 높다.

연속형 변수는 분포의 모양, 사분위 수, 평균, 표본편차를 확인할 수 있는데 plot을 그려서 더 확실하게 확인한다.

```
In [199]: mt = train[n_cols].melt()
g = sns.FacetGrid(mt, col='variable', col_wrap=4, sharex=False, sharey=False)
g.map(sns.kdeplot, 'value')
plt.show()
```



```
In [201]: sns.countplot(train[c_cols])
plt.show()
```



- 양의 왜도: 고객수, 강수량, 풍속, 총기압(분포 모양 교정 고려 시 log 화 필요)
- 음의왜도: 기온
- 정규분포: 습도
- 이산형 분포: 전문량, 중하층운량으로 0값이 가장 빈도가 높음
- 호선, 주말여부: A호선, 평일이 높은 비율

4-2 전체 년도 데이터에 대해 users와 날씨 정보에 대해 상관관계를 분석하고 주말여부에 따른 users 변수의 차이가 존재하는지 통계적으로 확인하라.

In [208]: #피어슨 상관관계를 통해 종속변수인 user와 날씨정보(연속형)에 대한 상관관계 분석

```
from scipy.stats import pearsonr

n_cols = train.select_dtypes('number').columns[2:-1]
for c in n_cols:
    coef, pv = pearsonr(train['users'], train[c])
    print(f"{c}-user 간 피어슨 상관계수: {coef:.3f} /pvalue({pv:.3f})")
```

기온-user 간 피어슨 상관계수: 0.030 /pvalue(0.000)
강수량-user 간 피어슨 상관계수: -0.002 /pvalue(0.069)
풍속-user 간 피어슨 상관계수: 0.050 /pvalue(0.000)
습도-user 간 피어슨 상관계수: -0.049 /pvalue(0.000)
증기압-user 간 피어슨 상관계수: -0.001 /pvalue(0.176)
현지기압-user 간 피어슨 상관계수: -0.011 /pvalue(0.000)
전운량-user 간 피어슨 상관계수: -0.008 /pvalue(0.000)
중하층운량-user 간 피어슨 상관계수: -0.011 /pvalue(0.000)
지면온도-user 간 피어슨 상관계수: 0.021 /pvalue(0.000)

피어슨 상관관계 분석결과, 강수량, 증기압이 통계적으로 유의하지 않았으며(유의확률 5% 하)
나머지 변수도 통계적으로 크게 유의미한 상관관계가 없다.

주말여부에 따른 users 변수 차이가 존재하는지 확인하기 위해 먼저 정규성 여부를 확인 후 정규성을 만족한다면 2 독립변수 ttest 를 만족하지 않는다면 맨휘트니 검정을 수행한다.(사실 데이터 수가 충분하므로 중심극한의 정리에 따라 ttest 도 가능)

- 귀무가설: 주말 여부에 따른 유저수는 차이가 없다.
- 대립가설: 주말여부에 따른 유저수는 통계적으로 차이가 있다.

In [209]: gr = [(name, g['users']) for name, g in train.groupby(['weekday'])]

```
from scipy.stats import shapiro, mannwhitneyu

for name, g in gr:
    s, p = shapiro(g)
    print(f"{name} 그룹 정규성 검정결과: {p:.3f}")
```

0 그룹 정규성 검정결과: 0.000
1 그룹 정규성 검정결과: 0.000

모든 그룹에서 귀무가설을 기각하므로 정규성을 만족하지 못한다. 따라서 맨휘트니 검정을 수행한다.

In [210]: s, p = mannwhitneyu(gr[0][1], gr[1][1])
print(f"맨휘트니 검정결과: {p:.3f}")

맨휘트니 검정결과: 0.000

맨휘트니 u 검정결과 0으로 귀무가설을 기각한다. 따라서 주말여부에 따른 유저수는 통계적으로 차이가 있다.
이 뜻은 유저수를 예측하는 회귀모델에서 주말여부가 예측력에 영향을 줌을 뜻한다.

4-3 users를 예측하는 2가지 회귀모델을 학습하고 모델에 대해 평가하라 (선형회귀 모델은 1개 필수 포함)

일반적인 선형회귀와 비선형 특징까지 포착할 수 있는 배경기반 앙상블 모델인 랜덤포레스트 회귀를 사용한다..

모델 적합 후 성능척도는 r2_score, rmse를 사용한다.

In [211]: train.columns

Out[211]: Index(['날씨', '호선', '시간', 'users', '기온', '강수량', '풍속', '습도', '증기압', '현지기압',
'전운량', '중하층운량', '지면온도', 'weekday'],
dtype='object')


```
In [213]: from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import make_column_transformer
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error

train, test = train.sample(10000), test.sample(2000)

train_x, train_y = train.drop(columns=['날짜', 'users']), train['users']
test_x, test_y = test.drop(columns=['날짜', 'users']), test['users']

n_cols = [ '기온', '강수량', '풍속', '습도', '증기압', '현지기압', '전운량', '중하층운량', '지면온도' ]
c_cols = [ '호선', 'weekday' ]

train_x = pd.get_dummies(train_x, columns=c_cols, drop_first=True)
test_x = pd.get_dummies(test_x, columns=c_cols, drop_first=True)

pre = make_column_transformer(
    (Pipeline([('scr', StandardScaler())])), n_cols,
    remainder='passthrough'
)

lr = Pipeline([
    ('pre', pre),
    ('rg', LinearRegression())
])

rf = Pipeline([
    ('rg', RandomForestRegressor(random_state=42))
])

m_list = [('선형회귀', lr), ('랜덤포레스트', rf)]
d_list = [('train', train_x, train_y), ('test', test_x, test_y)]

rs = pd.DataFrame(columns = ['model', 'data', 'r2', 'rmse'])

for n1, model in m_list:
    model.fit(train_x, train_y)
    for n2, x, y in d_list:
        pred = model.predict(x)
        r2 = r2_score(y, pred)
        rmse = np.sqrt(mean_squared_error(y, pred))
        rs.loc[len(rs)] = [n1, n2, r2, rmse]
display(rs)
```

		model	data	r2	rmse
0		선형회귀	train	0.102915	2166.370062
1		선형회귀	test	0.088707	2358.677983
2		랜덤포레스트	train	0.799784	1023.445977
3		랜덤포레스트	test	0.203351	2205.327390

```
In [215]: 2358.677983/(test_y.max() - test_y.min())
```

```
Out [215]: 0.11663343633486624
```

두 모델 학습결과 전반적으로 랜덤 포레스트 회귀의 성능이 뛰어났다.
다만 랜덤포레스트는 과적합이 심하므로 일반화된 모델을 사용한다면 선형회귀가 적합할 것이다.

랜덤포레스트 테스트셋 기준으로 모델성능을 확인한다면 $r2_score$ 는 0.2로 종속변수 분산변동의 약 20%를 설명하며 약 10%의 종속변수 예측에 대한 표준오차를 가진다.

선형회귀 테스트 셋 기준으로 평가 시 $r2$ 설명력이 8.8%로 아주 낮으며 $rmse$ 는 0.116%의 종속변수 예측에 대한 표준오차를 가진다.

일반화모델인 선형회귀를 선택한다면 $r2$ 설명력을 높이기 위한 방안이 필요할 것이다.

통계 (40점)

데이터 설명

- 데이터 출처 : 자체 제작
- 데이터 링크 : ./data/s1.csv

- 데이터 설명 : A,B 두 공장의 특정 일자의 수율(%)데이터. 단, 불량률은 (100 - 수율)로 정의한다.

```
In [2]: d5 = pd.read_csv('https://raw.githubusercontent.com/Datamanim/datarepo/refs/heads/master/adp/33/data/s1.csv')
d5.head()
```

```
Out[2]:
```

	year	month	day	A	B
0	2023	Sep	7	56.9	42.3
1	2023	Feb	20	100.0	89.0
2	2023	Jun	15	19.3	59.0
3	2023	Apr	11	98.0	84.8
4	2023	Jul	8	89.5	63.8

```
In [3]: d5.isna().sum()
```

```
Out[3]: year      0
month    0
day      0
A         5
B         5
dtype: int64
```

```
In [4]: d5.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
#   Column  Non-Null Count  Dtype
---  ---
0   year    50 non-null       int64
1   month   50 non-null       object
2   day     50 non-null       int64
3   A       45 non-null       float64
4   B       45 non-null       float64
dtypes: float64(2), int64(2), object(1)
memory usage: 2.1+ KB
```

결측치가 존재한다. 통계문제이므로 보간 시 데이터 왜곡을 불러올 수 있으므로 삭제한다.
(보간을 위한 데이터도 아님)

```
In [5]: d5 = d5.dropna()
```

5-1 공장별 불량률 차이 평균, 표준편차를 제시하시오.

```
In [15]: g = [('A', d5['A']), ('B', d5['B'])]

for name, d in g:
    m = d.mean()
    sd = d.std()
    print(f"{name} 공장 평균/표준편차: {m} / {sd}")
```

```
A 공장 평균/표준편차: 75.7175 / 21.10496220056268
B 공장 평균/표준편차: 70.44250000000001 / 14.824700239110644
```

5-2 A,B 공장의 불량률 평균 차이가 존재하는지 확인하려 한다. 귀무,대립 가설을 설정하고 설명하라

두 공장의 불량률 평균 차 여부를 묻는 문제는 정규성을 만족한다면 2샘플 독립표본 t검정을, 만족하지 않는다면 맨휘트니 u 검정을 수행한다. 유의수준 5% 하에서 검정한다.

- 귀무가설: 두 공장의 불량률 차이는 없다.
- 대립가설: 두 공장의 불량률 차이는 있다.

5-3 A,B의 차이가 있는지 통계검정 하고 설명하라

```
In [18]: from scipy.stats import shapiro, ttest_ind, mannwhitneyu

for name, d in g:

    stat, p = shapiro(d)

    print(f"{name} 샤피로 윌크 정규성 검정 통계량/pvalue: {stat:.3f} / {p:.3f}")
```

A 샤피로 윌크 정규성 검정 통계량/pvalue: 0.921 / 0.008
B 샤피로 윌크 정규성 검정 통계량/pvalue: 0.978 / 0.615

a 공장에서 pvalue 0.008으로 귀무가설을 기각하여 정규성을 만족하지 못하므로 비모수 검정인 맨휘트니 검정을 수행한다.

```
In [19]: stat, p = mannwhitneyu(g[0][1], g[1][1])

print(f"맨휘트니 u 검정결과 통계량/pvalue: {stat:.3f} / pvalue: {p:.3f}")
```

맨휘트니 u 검정결과 통계량/pvalue: 969.500 / pvalue: 0.104

맨휘트니 u 검정결과 pvalue 0.104으로 유의수준 5% 하 귀무가설을 기각하지 못하므로 두 공장의 불량률 차이는 없다.

데이터 설명

- 데이터 출처 : <https://www.data.go.kr/data/15123586/fileData.do> (<https://www.data.go.kr/data/15123586/fileData.do>) 후처리
- 데이터 링크 : ./data/s2.csv
- 데이터 설명 : 일자별 지하철 노선별 지연 데이터

```
In [20]: import pandas as pd
d6 = pd.read_csv('https://raw.githubusercontent.com/Datamanim/datarepo/refs/heads/master/adp/33/data/s2.csv')
d6.head()
```

Out[20]:

	지연ID	지연일자	노선	최대지연시간
0	ID_0	2023-09-01	1호선	15분
1	ID_1	2023-09-01	1호선	15분
2	ID_2	2023-09-01	4호선	10분
3	ID_3	2023-09-01	4호선	10분
4	ID_4	2023-09-04	4호선	10분

```
In [21]: d6.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 833 entries, 0 to 832
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   지연ID      833 non-null    object
1   지연일자    833 non-null    object
2   노선        833 non-null    object
3   최대지연시간 833 non-null    object
dtypes: object(4)
memory usage: 26.2+ KB
```

6-1 노선에 상관없이 일별 최대 지연시간이 5~15분으로 발생하는 경우는 하나의 사건으로 보자. 해당 사건이 일자별 발생하는 빈도가 푸아송분포를 따르는지 확인하는 방법 2가지를 기술하고 결과를 보여라

- 푸아송 분포는 평균과 분산이 동일함을 가정한다. 따라서 분산비가 1인지 확인하여 푸아송 분포 여부를 확인한다.
- k-s 테스트는 누적분포확률을 비교하여 특정 데이터가 동일분포를 가지는지 확인하는 통계기법이다. 따라서 k-s테스트를 통해 해당 빈도가 푸아송 분포의 cdf와 동일한지 확인한다.

```
In [22]: d6['최대지연시간'].value_counts()
```

```
Out[22]: 10분      414
          5분      179
          15분     153
          20분      58
          25분      16
          30분 이상   13
          Name: 최대지연시간, dtype: int64
```

```
In [28]: tmp = d6.loc[d6['최대지연시간'].isin(['5분', '10분', '15분']), '최대지연시간'].value_counts().reset_index().set_index('index')
```

```
In [29]: tmp
```

```
Out[29]:
```

최대지연시간	
index	
10분	414
5분	179
15분	153

```
In [40]: tm = tmp.var(ddof=1)/tmp.mean().values
```

```
print(f"분산비: {tm[0]}")
```

분산비: 83.12466487935657

```
In [47]: from scipy.stats import kstest
```

```
stat, p = kstest(tm, 'poisson', args=(tmp.mean(), ))
print(f"ks-test 검정통계량/pvalue : {stat:.3f} / pvalue: {p:.3f}")
```

ks-test 검정통계량/pvalue : 1.000 / pvalue: 0.000

분산비가 83.12466487935657로 1보다 확연히 크며 ks-test 검정결과 pvalue: 0.000으로 귀무가설을 기각하므로 해당 분포는 포아송 분포를 따르지 않는다.

6-2 10분이하, 10분초과 -20분 이하, 20분초과 3가지로 지연시간을 지연정도로 분류하라. 호선별 지연정도를 교차표로 구하여라

```
In [50]: cond1 = d6['최대지연시간'].isin(['5분', '10분'])
cond2 = d6['최대지연시간'].isin(['15분', '20분'])

d6['구분'] = np.where(cond1, 1, np.where(cond2, 2, 3))

cr = pd.crosstab(d6['노선'], d6['구분'])

display(cr)
```

구분	1	2	3
노선			
1호선	269	67	16
4호선	324	144	13

다음과 같이 교차표를 생성하였다.

6-3 지하철 호선별로 지연정도가 차이가 있는지 확인하려한다. 귀무가설과 대립가설을 설정한 후 통계 검정을 실시하라

노선과 지연정도는 모두 질적변수(카테고리변수)이다. 따라서 두 변수에 대한 차이가 있는지 여부는 비모수검정인 카이제곱 독립성 검정을 통해 독립적인지 연관이 있는지 확인한다.

유의수준 5% 하에서 검정을 시행한다.

- 귀무가설: 지하철 호선과 지연정도는 독립적이다.(차이가 없다.)
- 대립가설: 지하철 호선에 따른 지연정도는 연관이 있다.(차이가 있다.)

```
In [51]: from scipy.stats.contingency import chi2_contingency

stats, p, ddof, expected = chi2_contingency(cr)

print(f"카이제곱 독립성 검정 통계량/pvalue: {stat:.3f} / pvalue: {p:.3f}")
```

카이제곱 독립성 검정 통계량/pvalue: 1.000 / pvalue: 0.001

유의수준 5% 하에서 pvalue: 0.001로 귀무가설을 기각할 수 있다.
따라서 지하철 호선에 따른 지연정도는 연관이 있다.는 대립가설은 통계적으로 유의하다.

데이터 설명

- 데이터 출처 : 자체제작
- 데이터 링크 : ./data/s3.csv
- 데이터 설명 : 시계열 정보를 가지는 250행의 데이터

```
In [52]: import pandas as pd
d7= pd.read_csv('https://raw.githubusercontent.com/Datamanim/datarepo/refs/heads/master/adp/33/data/s3.csv'W
            , header=None)
d7.columns = ['time']
d7.head()
```

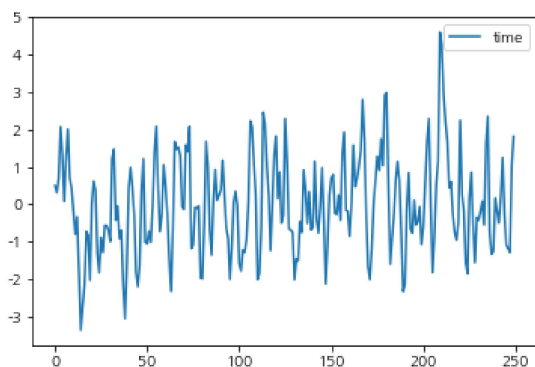
Out[52]:

	time
0	0.496714
1	0.308778
2	0.692133
3	2.070861
4	1.326845

7-1 ar ma arma에 대해 설명하고, ACF, PACF는 패턴을 통해 설명하라.

1. AR(p): 현재시점의 값이 과거 자신들의 값들의 선형 결합으로 표현되는 모델(정상성 가정)
p 파라미터를 가지며 ACF 그래프 상에서는 값이 천천히 떨어지거나 사인 파형을 그리고 PACF에서는 lag p 시점에서 cutoff 발생
2. ma(q): 현재 시점의 값이 과거의 오차항들의 선형결합으로 표현되는 모델(정상성 가정) q 파라미터를 가지며, ACF 그래프 상에서는 lag q 시점에서 cutoff(신뢰구간 전 급격히 떨어지는 점), pacf 그래프에서는 값이 천천히 떨어지거나 사인 파형을 그림
3. arma(p, q): 자기회귀와 이동평균 요소를 결합한 모델로 정상성 가정 pacf(p), acf(q) 에서 각각 해당 차수에서 cutoff 발생

```
In [55]: sns.lineplot(data=d7)
plt.show()
```



```
In [58]: from statsmodels.tsa.stattools import adfuller

rs = adfuller(d7)

print(f"adfuller 정상성 확인결과 통계량/pvalue: {rs[0]:.3f} / {rs[1]:.3f}")
```

adfuller 정상성 확인결과 통계량/pvalue: -5.615 / 0.000

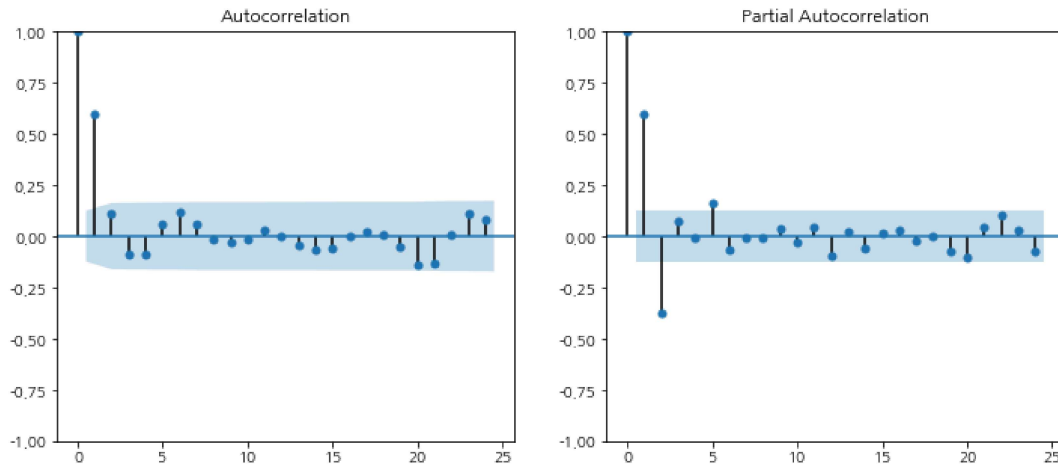
adfuller 정상성 확인(유의수준 5% 하 가설검정)

- 귀무가설: 단위근이 있다.(정상성 불만족)
- 대립가설: 단위근이 없다.(정상성 만족)

7-2 Acf pacf를 그리고 해석하라, 해당 Plot data를 기반으로 ARMA(p,q)모형 제시하라

```
In [60]: from statsmodels.graphics.tsaplots import plot_pacf, plot_acf

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
plot_acf(d7, ax=ax[0])
plot_pacf(d7, ax=ax[1])
plt.show()
```



p 파라미터: pacf 그래프에서 lag2에서 cutoff 발생 후 대체로 신뢰구간 내 point 존재로 p는 2 q 파라미터: acf 그래프에서 lag1에서 cutoff 발생 후 대체로 신뢰구간 내 존재, q=1

따라서 ARMA(2, 1)을 추천한다.

```
In [63]: from statsmodels.tsa.arima.model import ARIMA

model = ARIMA(d7, order=(2, 0, 1)).fit()

print(model.summary())
```

SARIMAX Results						
Dep. Variable:	time	No. Observations:	250			
Model:	ARIMA(2, 0, 1)	Log Likelihood	-345.443			
Date:	Tue, 14 Oct 2025	AIC	700.887			
Time:	10:05:51	BIC	718.494			
Sample:	0	HQIC	707.973			
	- 250					
Covariance Type:	opg					
	coef	std err	z	P> z	[0.025	0.975]
const	-0.0051	0.122	-0.042	0.966	-0.244	0.233
ar.L1	0.6199	0.172	3.613	0.000	0.284	0.956
ar.L2	-0.2538	0.120	-2.115	0.034	-0.489	-0.019
ma.L1	0.2280	0.168	1.356	0.175	-0.102	0.558
sigma2	0.9256	0.075	12.379	0.000	0.779	1.072
Ljung-Box (L1) (Q):		0.00	Jarque-Bera (JB):	8.74		
Prob(Q):		0.99	Prob(JB):	0.01		
Heteroskedasticity (H):		1.15	Skew:	0.33		
Prob(H) (two-sided):		0.54	Kurtosis:	3.63		

Warnings:

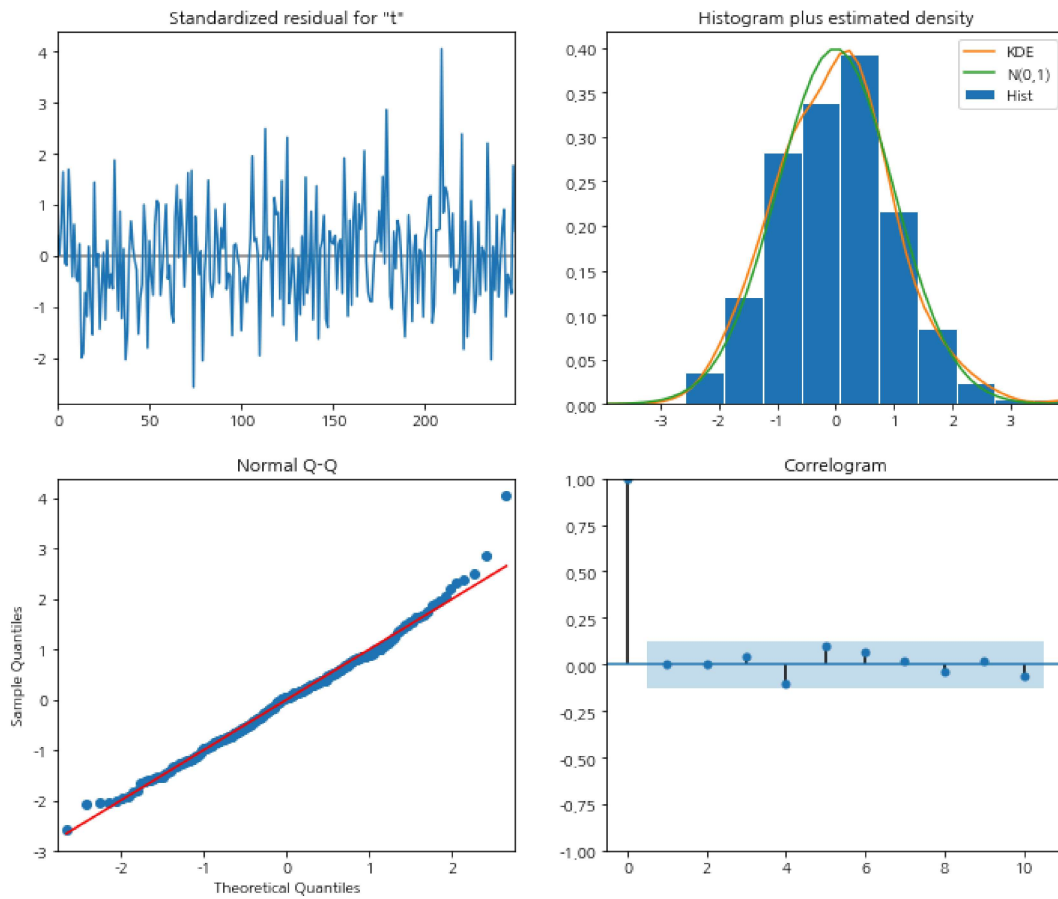
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

로그 우도: -345.443로 낮음(높을수록 좋음)

AIC 700.887

정규성(JB): 0.01로 만족하지 않음 등분산성: 0.54로 만족 독립성(lb): 0.99로 만족

```
In [66]: model.plot_diagnostics(figsize=(12, 10))  
plt.show()
```



```
In [ ]:
```